GUIA TÉCNICO | 14

CNNs na prática: MNIST, Fashion-MNIST e CIFAR

Capítulo 14/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Construir uma CNN compacta e distinguir domínios de imagens.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

MNIST é relativamente simples e em escala de cinza; Fashion-MNIST e CIFAR exigem representações progressivamente mais complexas. Normalizar pixels com os dados corretos e respeitar canais (H,W,C). Treinos reais ficam no laboratório opcional.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: X∈R^(B,H,W,C)

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Uma CNN só demonstra aprendizado quando rótulos correspondem a padrões e o desempenho é medido fora do treino.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
rng=np.random.default_rng(15)
X=rng.random((8,28,28,1),dtype=np.float32)
flat=X.reshape(len(X),-1)
print('Batch de imagens:',X.shape,'flatten:',flat.shape)
assert flat.shape==(8,784)
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Classificador de imagens sintéticas com padrão aprendível

Uma CNN só demonstra aprendizado quando rótulos correspondem a padrões e o desempenho é medido fora do treino.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
rng=np.random.default_rng(14);  ↪
    n=260;X=np.zeros((n,12,12),dtype=np.float32);y=np.repeat([0,1],n//2)
for i in range(n):
    if y[i]==0:X[i,:,4:6]=1.0
    else:X[i,4:6,:]=1.0
    X[i]+=rng.normal(0,.22,size=(12,12))
idx=rng.permutation(n);X,y=X[idx],y[idx]
Xt,Xe,yt,ye=train_test_split(X.reshape(n,-1),y,test_size=.25,stratify=y,random_state=14)
m=LogisticRegression(max_iter=300).fit(Xt,yt)
a=accuracy_score(ye,m.predict(Xe))
print('Baseline linear para listras horizontais/verticais:',round(a,3));assert a>.8
```

Este experimento testa padrões com baseline linear. CNN verdadeira deve usar amostras de treino e teste separadas e métricas reais, não ruído com rótulos arbitrários.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Por que não usar Flatten imediatamente em imagens grandes?

Resposta: Porque perde estrutura espacial e eleva muito a quantidade de parâmetros em camadas densas.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.

• Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide